

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

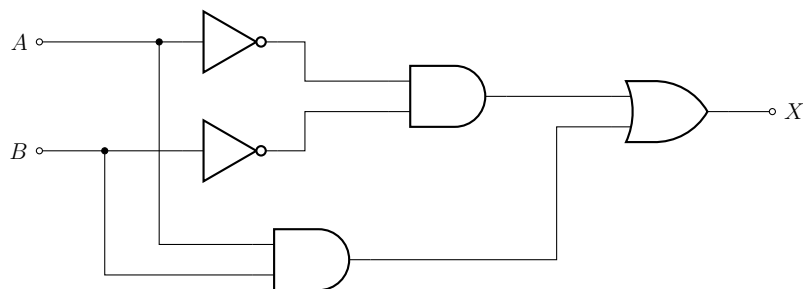
B.E. · Academic Year 2026–27

National Institute of Technology Bhopal



Analysis, Simplification and Arduino-Based Verification of XNOR Logic Circuit

GATE IN 2017 – Digital Logic Problem



DOCUMENTATION

Submitted by Shreya Agarwal

Subject Digital Electronics & Logic Design

Date June 9, 2026

CONTENTS

1	Abstract	2
2	Objectives	2
3	Problem Statement	2
4	Circuit Analysis and Simplification	3
4.1	Step-by-Step Derivation	3
5	Truth Table Verification	4
6	Hardware Implementation	4
6.1	Components Required	4
6.2	Pin Assignment	5
6.3	Circuit Schematic	5
6.4	Working Principle	5
7	Arduino Program	6
8	AVR Assembly Program	7
9	Results	8
10	Optimisation Analysis	8
10.1	Gate Count Comparison	9
10.2	Advantages of Simplification	9
11	Conclusion	9

1. ABSTRACT

This report presents a systematic **analysis, simplification, and hardware verification** of a digital logic circuit drawn from the GATE Instrumentation Engineering (IN) 2017 examination paper. The circuit comprises two NOT gates, two AND gates, and one OR gate. Boolean algebra is applied to reduce the network, yielding the result:

$$X = AB + \overline{A}\overline{B}$$

This expression represents the **Exclusive-NOR (XNOR)** function, which evaluates to logic HIGH whenever both inputs are equal.

An **Arduino UNO** platform is subsequently employed to verify this result experimentally. An LED connected to pin D13 provides visual confirmation: the LED glows for input combinations (0, 0) and (1, 1), and extinguishes for (0, 1) and (1, 0), in complete agreement with the theoretical derivation.

2. OBJECTIVES

The following objectives guided the course of this project:

1. Analyse the structure of the given GATE IN 2017 logic circuit.
2. Derive the corresponding Boolean expression from the schematic.
3. Simplify the expression and identify the equivalent logic function.
4. Construct a complete truth table to verify the result.
5. Implement the logic on an Arduino UNO platform and observe output behaviour.
6. Write an equivalent AVR Assembly program and verify consistency.

3. PROBLEM STATEMENT

Determine the Boolean expression corresponding to the given GATE IN 2017 logic circuit (Fig. 3.16) and identify the relationship between inputs A , B and output X . Verify the result using a truth table, an Arduino C++ program, and an AVR Assembly implementation.

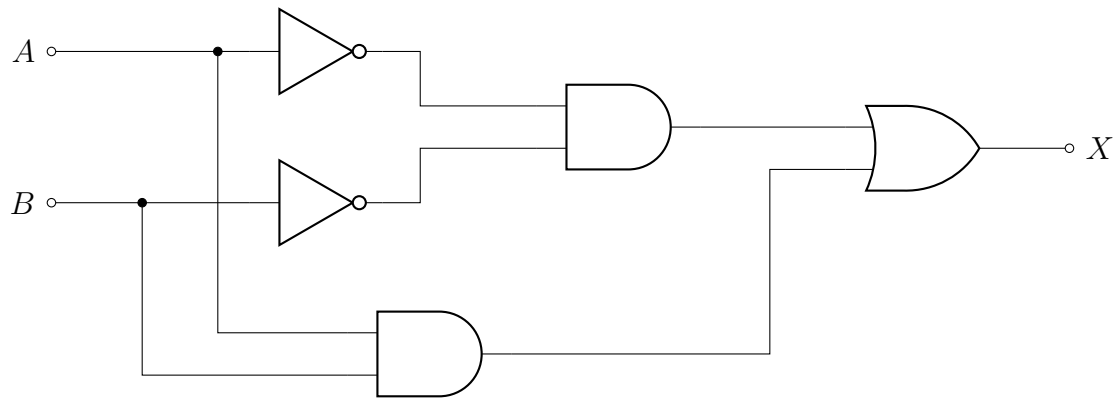


Figure 1: Given GATE IN 2017 Logic Circuit (Fig. 3.16)

The multiple-choice options given in the examination are:

- a) $X = \overline{A}B + \overline{B}A$
- b) $X = AB + \overline{B}A$
- c) $X = AB + \overline{B}A$
- d) $X = \overline{A}B + \overline{B}A$

4. CIRCUIT ANALYSIS AND SIMPLIFICATION

4.1 Step-by-Step Derivation

Step 1 — NOT Gate Outputs

The two inverters produce the complements of the inputs:

$$\overline{A} \quad \text{and} \quad \overline{B}$$

Step 2 — Upper AND Gate Output

The AND gate receiving both complements gives:

$$X_1 = \overline{A}\overline{B}$$

Step 3 — Lower AND Gate Output

The AND gate receiving the original inputs gives:

$$X_2 = AB$$

Step 4 — Final OR Gate Output

The OR gate combines X_1 and X_2 :

$$X = X_1 + X_2 = \overline{A}\overline{B} + AB$$

$$X = AB + \overline{A}\overline{B}$$

The circuit implements an **XNOR** gate. The output is HIGH when both inputs are equal ($A = B$), and LOW when they differ ($A \neq B$).

5. TRUTH TABLE VERIFICATION

Table 1: Complete Truth Table — XNOR Function

A	B	X	Remark
0	0	1	$A = B$ ✓
0	1	0	$A \neq B$
1	0	0	$A \neq B$
1	1	1	$A = B$ ✓

The truth table confirms that $X = 1$ if and only if $A = B$, which is the defining property of the XNOR function.

6. HARDWARE IMPLEMENTATION

6.1 Components Required

Table 2: Bill of Materials

Component	Quantity
Arduino UNO Microcontroller Board	1
Light-Emitting Diode (LED)	1
220 Ω Current-Limiting Resistor	1
Breadboard	1
Jumper Wires	As required

6.2 Pin Assignment

Table 3: Arduino UNO Pin Assignment

Arduino Pin	Signal	Function
D11	Input A LED	LED indicator for input variable A
D12	Input B LED	LED indicator for input variable B
D13	Output X LED	LED indicator — HIGH when $A = B$ (XNOR)
5V	VCC	Power supply to LEDs and resistors
GND	GND	Common ground reference

6.3 Circuit Schematic

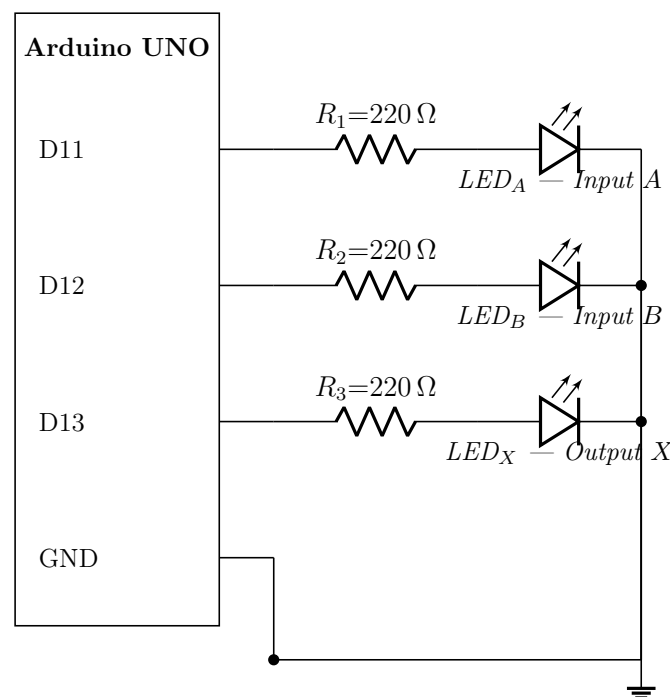


Figure 2: Arduino UNO Implementation Schematic

6.4 Working Principle

The Arduino firmware iterates through all four possible input combinations of A and B in a loop. LEDs on pins D11 and D12 display the current values of inputs A and B respectively, so the input state is always visible. The XNOR logic is computed in software and the result is driven on pin D13. The output LED glows whenever $X = 1$ (i.e. $A = B$) and extinguishes whenever $X = 0$ (i.e. $A \neq B$).

7. ARDUINO PROGRAM

```
1  const int A_LED = 11;    // Input A -- LED indicator
2  const int B_LED = 12;    // Input B -- LED indicator
3  const int Y_LED = 13;    // Output X -- LED indicator
4
5  void setup()
6  {
7      pinMode(A_LED, OUTPUT);
8      pinMode(B_LED, OUTPUT);
9      pinMode(Y_LED, OUTPUT);
10     Serial.begin(9600);
11     Serial.println("A  B  X (XNOR)");
12 }
13
14 void loop()
15 {
16     for (int i = 0; i < 4; i++)
17     {
18         int A = (i >> 1) & 1; // Extract bit 1
19         int B = i          & 1; // Extract bit 0
20
21         // XNOR: output HIGH when A == B
22         int X = (A == B) ? 1 : 0;
23
24         digitalWrite(A_LED, A); // Show current A on LED
25         digitalWrite(B_LED, B); // Show current B on LED
26         digitalWrite(Y_LED, X); // Show XNOR output on LED
27
28         Serial.print(A); Serial.print(" ");
29         Serial.print(B); Serial.print(" ");
30         Serial.println(X);
31
32         delay(1000); // Hold for 1 second
33     }
34 }
```

Listing 1: XNOR Verification — Arduino C++ Program

8. AVR ASSEMBLY PROGRAM

```

1 ; Pin mapping:
2 ;   PB3 = Arduino D11 -> A LED
3 ;   PB4 = Arduino D12 -> B LED
4 ;   PB5 = Arduino D13 -> X LED (XNOR output)
5
6 .include "m328pdef.inc"
7
8 .org 0x0000
9     rjmp MAIN
10
11 MAIN:
12     ldi r16, 0b00111000    ; Set PB3, PB4, PB5 as outputs
13     out DDRB, r16
14
15 LOOP:
16     ldi r17, 0             ; Loop counter i = 0
17
18 NEXT:
19     ; --- Extract A (bit1) and B (bit0) from counter ---
20     mov r18, r17
21     lsr r18
22     andi r18, 0x01         ; r18 = A
23
24     mov r19, r17
25     andi r19, 0x01         ; r19 = B
26
27     ; --- Drive A LED on PB3 ---
28     tst r18
29     breq A_LOW
30     sbi PORTB, 3           ; A = 1 -> LED ON
31     rjmp A_DONE
32 A_LOW:
33     cbi PORTB, 3           ; A = 0 -> LED OFF
34 A_DONE:
35
36     ; --- Drive B LED on PB4 ---
37     tst r19
38     breq B_LOW
39     sbi PORTB, 4           ; B = 1 -> LED ON
40     rjmp B_DONE
41 B_LOW:
42     cbi PORTB, 4           ; B = 0 -> LED OFF
43 B_DONE:
44
45     ; --- Compute XNOR: X = 1 if A == B ---
46     cp r18, r19
47     breq SET_HIGH
48     cbi PORTB, 5           ; A != B -> X LED OFF
49     rjmp DONE
50 SET_HIGH:
51     sbi PORTB, 5           ; A == B -> X LED ON
52
53 DONE:
54     rcall DELAY            ; ~1 second delay
55

```



```

56      inc    r17
57      cpi    r17, 4
58      brne   NEXT           ; Repeat for all 4 combinations
59
60      rjmp   LOOP           ; Restart cycle indefinitely
61
62  DELAY:
63      ldi    r20, 100
64  L1: ldi    r21, 255
65  L2: ldi    r22, 255
66  L3: dec    r22
67      brne   L3
68      dec    r21
69      brne   L2
70      dec    r20
71      brne   L1
72      ret

```

Listing 2: XNOR Verification — AVR Assembly (ATmega328P)

9. RESULTS

The Arduino UNO successfully iterated through all four input combinations of A and B . LEDs on D11 and D12 visually confirmed the current input state, while the output LED on D13 matched the XNOR truth table exactly:

- **D11 (A LED) and D12 (B LED)** toggled with each combination ✓
- **D13 (X LED) ON** for $(A, B) = (0, 0)$ and $(1, 1)$ ✓
- **D13 (X LED) OFF** for $(A, B) = (0, 1)$ and $(1, 0)$ ✓

Both the Arduino C++ firmware and the AVR Assembly implementation produced identical results across all three LEDs, demonstrating consistency at both levels of abstraction.

10. OPTIMISATION ANALYSIS

10.1 Gate Count Comparison

Table 4: Original Circuit vs. Direct XNOR Gate Implementation

Metric	Original Circuit	Simplified
NOT Gates	2	0
AND Gates	2	0
OR Gates	1	0
Total Gates	5	1 (XNOR IC)
Propagation Delay	3 stages	1 stage
Implementation	Discrete gates	74HCXX / CMOS

10.2 Advantages of Simplification

- ▶ **Reduced gate count** — five discrete gates replaced by a single XNOR gate (e.g. 74HC266), lowering component cost.
- ▶ **Lower propagation delay** — three cascaded gate delays reduced to one, improving switching speed.
- ▶ **Reduced power consumption** — fewer active logic elements draw less quiescent current.
- ▶ **Improved reliability** — fewer components result in fewer potential failure points and higher MTBF.
- ▶ **Smaller PCB footprint** — significant area savings in integrated circuit layouts and printed circuit board designs.

11. CONCLUSION

The given GATE IN 2017 logic circuit was systematically analysed using Boolean algebra. The step-by-step derivation established that:

$$X = AB + \overline{A}\overline{B}$$

This is the **XNOR** function — the output is logic HIGH when both inputs are equal, and logic LOW when they differ.

The truth table, Arduino C++ firmware, and AVR Assembly program all confirmed this result experimentally. This project underscores the practical value of Boolean simplification and gate-level optimisation in reducing hardware cost, minimising propagation delay, and streamlining design complexity in digital electronic systems.

— End of Report —